

Process log

WHAT I HAVE DONE AS PROCESS

1. **Read the packet properly, then split it** — five files saved verbatim, untouched; a second copy for reconciling.
2. **Reconciled the three documents** — ICP into one table; Campaign Manager schema rebuilt from the stub one field at a time after a first pass that over-encoded the CEO note; R-17 read as FS-only, record-before-action. Every decision carries its reason.
3. **Built the artifact skeleton** — six MCP tools (fetch, diff, lookup, audit, kill check, emit), a seven-check eval, playbook instances. `emit_brief` is the only way a brief becomes a file; `@audited` makes the render unreachable until the R-17 record is written.
4. **Wrote the two skills** — `regulatory-brief`: a trigger becomes a three-section brief, every sentence cited, ≤ 250 words, one ask, applicability conditional. `regulatory-watch`: one check cycle — diff the sources, judge material vs noise, screen accounts, call the brief skill once per affected account, emit structured JSON.
5. **Built the watch layer** — the agent runs on a set interval, `diff_source` is its memory, the server schedules and records. First cycle failed on an `mcp` 2.x rename; the agent reported `error` rather than inventing a verdict.
6. **Built a dashboard to operate it** — regulation toggles, manual checks, live phase, approval, kill, identity switching, all on localhost.
7. **Tested whether it actually catches a diff and writes the right brief** — the live documents had not changed, so Wayback Machine captures became baselines: a test mode with two scenarios replays changes that really happened. The noise case came back noise; the material case produced a brief.
8. **Connected HubSpot** — a CRM table (mock first, then real Canadian institutions), the HubSpot MCP read-only inside the headless agent, `applies_to` per trigger and a fixed screening order; one change fans out to many accounts, one brief each, a `score` and a `create` record each.
9. **Fixed the dashboard's bugs** — two regulations starting in the same second collided on timestamp; CSS overrode `hidden` so the test banner would not close; scenario A had to drop SR 26-2's own snapshots to be a genuine replay.

WHAT FAILED

- **The first SR 26-2 brief took three emits.** 262 words against a 250 cap; then a source hash that would not match because `read_text()` normalised `\r\n` after I had hashed the raw text; then the eval itself crashed — a set passed into `sum()` — after printing seven PASS lines. Each failure left its own audit record, which is correct.
- **There was no honest way to test detection.** The documents I watch had not changed and would not on my schedule. Looking for past versions to seed a baseline, the

Wayback Machine fought back at every step: the availability API returned 429; the direct capture URL handed me Wayback's HTML wrapper instead of the PDF; some captures came back gzip-compressed; and a fixture built from a non-`id_` capture carried rewritten links that all looked new.

- **Test mode would not switch off.** The banner and scenario chooser stayed on screen after Exit — `display:flex` in my own CSS was overriding the `hidden` attribute.
- **Operating the dashboard found what the code review did not.** Resetting `last_run` to re-test made the scheduler treat both watches as due, and I stopped the server inside its 30-second tick — a never-run watch now waits for a manual first check. "Check all" gave both regulations the same timestamp, so the check page keyed on timestamp alone opened DORA's record from SR 26-2's row; keyed by regulation + timestamp now.

WHAT I LEARNED

- **The hash is the detector; the agent is the judge.** A hash-only watcher would have written a brief off 149 links that merely changed format. The agent read the diff, saw the page text was identical, called it noise and said why. Neither half works alone.
- **Constraints belong in structure, not in the prompt.** "Call `write_audit` first" is a request. `@audited` — render code unreachable until the record is written — is a fact, and eval check 5 verifies it independently. Two layers, one of them not the model's to skip.
- **An agent's honesty can be designed.** When the MCP failed to connect, the agent returned `error` with the mechanism rather than a verdict; when my snapshot logic was wrong, it named the bug in its noise judgment. Both because the skill demands the *why*, not just the result.
- **Keep the noise.** A verdict overwrites the baseline; a wrong "noise" is never raised again. The reason and the raw diff stay on the check page so a miss can at least be seen by a person.

WITH ANOTHER THREE HOURS

High - Brief credibility: an LLM-judge pass that checks each sentence against the source it cites; a rule that the *one ask* may only request an action inside the bank; a whitelist of what in `reign_known_facts` may appear in a brief. - Approver rule: must be a person flagged `can_approve`, never the operator running the agent. - `send_message`: an `@audited` tool that writes the R-17 `message` record, checks `approved_at` and consent, and in this environment writes an `.eml` to disk instead of sending.

Mid - Noise review queue: a "disagree" action on a noise verdict that re-queues the check and keeps the old baseline. - New document → proposed trigger: when an index page shows a material new document, draft a trigger for a human to accept instead of writing a brief under the wrong playbook.

Low - Triggers for the other target industries: FDA PCCP (biopharma, non-FS path), CMMC (defense, monitor only), EU AI Act. - Company-level impact scoring: Clay enrichment of public company signals wired in through MCP during screening, each enrichment written as an R-17 `enrich` record, briefs ordered by score.

TWO THINGS AS THEY ACTUALLY RUN

The screening step, from `artifact/skill/regulatory-watch/SKILL.md`. The agent's judgment about who is affected is an ordered rule, not a free reading of the CRM:

```
4. Screen accounts (HubSpot). `search_crm_objects` on companies, requesting the
`reign_*`
  properties plus `name`. For every company decide, in this order, and record one
  reason:
    - in `icp.excluded` (e.g. Mid-market SaaS)                → excluded: "segment excluded
    by CEO"
    - `reign_segment` not in `trigger.applies_to.segments` → excluded: "not a segment
    this regulation reaches"
    - fails an ICP gate                                       → excluded: name the gate
    - fails `trigger.applies_to.requires`                    → excluded with the specific
    reason
                                                                ("US operations under
    $30B – SR 26-2 not expected to apply")
    - otherwise                                              → affected.
  For each affected account whose `reign_regulated_fs` is true, write a `score`
  record
  (`write_audit`, action "score", object = reign_account_id, object_ref = the HubSpot
  id).
  If it raises, that account is excluded with reason "audit record refused".
```

How the server starts the agent, from `artifact/server/app.py`. Two tool classes in one run; the agent may read the CRM and may never write to it:

```
ALLOWED = ["mcp__reign-tools__diff_source", "mcp__reign-tools__fetch_source",
"mcp__reign-tools__lookup",
            "mcp__reign-tools__check_kill", "mcp__reign-tools__emit_brief",
"mcp__reign-tools__write_audit", "Read",
            # HubSpot is the system of record (CEO note). Read-only tools only; the
            agent never writes to the CRM.
            "mcp__claude_ai_HubSpot__search_crm_objects",
"mcp__claude_ai_HubSpot__get_crm_objects",
            "mcp__claude_ai_HubSpot__query_crm_data",
"mcp__claude_ai_HubSpot__get_properties",
            "mcp__claude_ai_HubSpot__search_properties",
"mcp__claude_ai_HubSpot__tool_guidance"]

cmd = [CLAUDE, "-p", build_prompt(t), "--output-format", "stream-json", "--verbose",
        "--mcp-config", str(mcp_config_path()), # reign-tools; the account-level
        HubSpot connector loads alongside
```

```
    "--allowedTools", *ALLOWED, "--max-turns", "60"]  
    env = {**os.environ, "REIGN_RUNS_DIR": str(P().RUNS)}    # normal vs test state tree
```

`build_prompt` is the two skill files inlined verbatim plus the trigger and playbook paths.
The server reads the stream one event at a time and maps each tool call to the phase shown on the dashboard.